

AgentConductor: 多智能体竞赛级代码生成的拓扑演化

Siyu Wang¹ Ruotian Lu¹ Zhihao Yang¹ Yuchao Wang¹ Yanzhou Zhang¹ Lei Xu¹ Qimin Xu¹
Guojun Yin² Cailian Chen^{1†} Xinping Guan^{1†}

摘要

由大语言模型（LLM）驱动的多智能体系统（MAS）通过预定义交互拓扑协调专门化智能体，在竞赛级代码生成等复杂任务中展现出潜力。近期研究表明，精心设计的多智能体工作流和通信图能够利用协作推理，显著提高代码生成性能。然而，现有方法既不能根据任务难度调整拓扑密度，也不能利用执行反馈在单个实例内部迭代精炼拓扑，因而造成冗余通信和性能瓶颈。为解决这些问题，我们提出 AgentConductor：一种以 LLM 编排器智能体为核心、经强化学习优化的 MAS，支持端到端、反馈驱动的交互拓扑动态生成。对于每个问题，AgentConductor 推断智能体角色和任务难度，然后构造适应该任务、感知密度的分层有向无环图（DAG）拓扑；这一过程由两项关键创新支撑。第一，我们设计一种新的拓扑密度函数，以数学方式刻画多智能体交互中的通信特征。第二，我们采用难度区间划分，避免过度剪枝，精确测量各难度等级下的拓扑密度上界，并实现更细粒度的控制。在三个竞赛级和两个基础代码数据集上的实证结果表明，AgentConductor 达到当前最佳准确率，相比最强基线，pass@1 准确率最高提升 14.6%，密度最高降低 13%，词元成本最高降低 68%。

[†] 通讯作者 ¹ 上海交通大学 ² 美团。 通讯作者：Cailian Chen <cailianchen@sjtu.edu.cn>, Xinping Guan <xpguan@sjtu.edu.cn>。

1. 引言

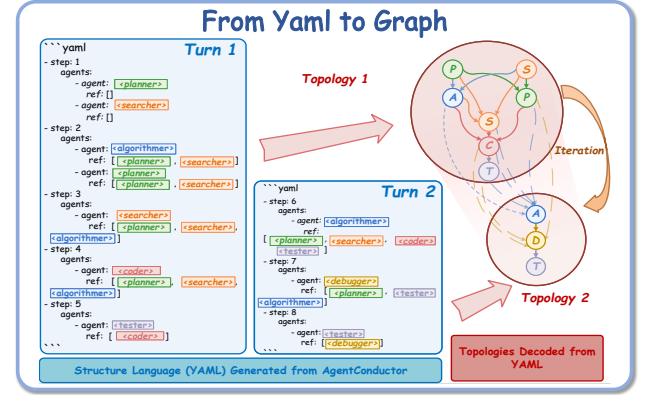


图 1. 拓扑的 YAML 表示、到实际图的映射，以及两轮图演化。

竞赛级编程被广泛视为要求最高的问题求解任务之一 (Khan et al., 2023; Hendrycks et al., 2021)。这类问题覆盖多个难度等级，其中一些实例接近竞赛难度的上限。解决更具挑战性的实例，需要深入理解题意、具备复杂推理能力和扎实的算法专长。近期，基于 LLM 的 MAS 在竞赛级代码生成上取得了显著进展 (Islam et al., 2024; 2025)。其性能提升主要来自精心设计的交互拓扑：这些拓扑促进高效协作并提高解答准确率。然而，这类系统通常依赖固定拓扑。固定设计可以提高困难实例的性能上限，但在处理简单实例时会引入冗余交互和不必要的计算开销。为缓解复杂协作导致的成本增长，一些方法 (Zhang et al., 2024a; Wang et al., 2025b) 针对某类问题优化并剪枝拓扑，另一些方法则生成以问题为条件的交互结构 (Zhang et al., 2024b)。尽管这些方法通过有限的动态拓扑优化降低了成本，但它们不会调整拓扑密度以匹配任务难度，也无法在单个问题内部利用执行反馈迭代精炼拓扑，而这些能力对竞赛级代码生成至关重要。由此产生一个核心问题：如何自动生

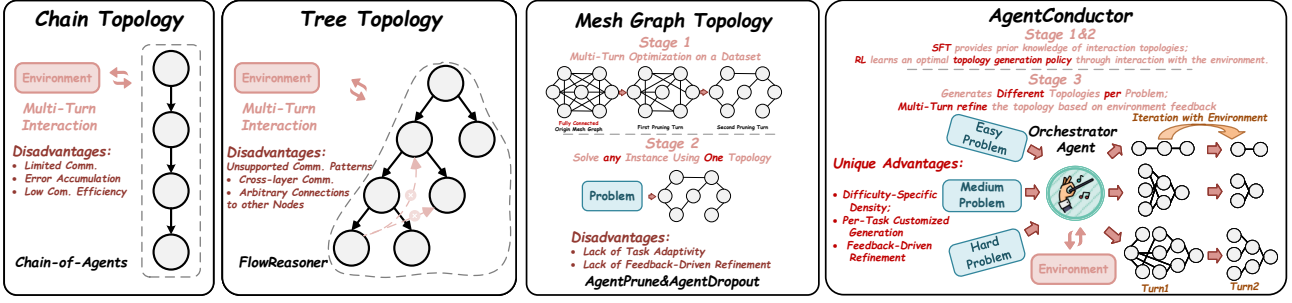


图 2. 本文方法与经典基线在拓扑结构和优化范式上的对比

成任务专用交互拓扑，使其密度随难度扩展，并能响应执行反馈持续演化？

现有方法尝试通过剪枝或生成动态优化交互拓扑。图剪枝方法 (Zhang et al., 2024a; Zhuge et al., 2024) 通过迭代移除边或角色降低成本。然而，剪枝拓扑一旦得到，通常会在同一数据集或基准的不同问题实例间复用。因此，这类方法可能无法匹配任务专用需求，并导致性能下降。图生成方法 (Zhang et al., 2024b) 以输入问题为条件构造拓扑，是对剪枝方法的改进；与剪枝相比，它们支持实例级拓扑生成。但生成的拓扑在单个问题内部仍然保持冻结，不能适应执行反馈。此外，两类方法通常都依赖单调稀疏性约束，促使拓扑收敛到固定密度区间。不同于上述方法，以工作流为中心的 RL 方法 (Gao et al., 2025; Li et al., 2025) 通过端到端 RL 训练单一智能体管理线性化多阶段工作流，支持基于环境反馈的多轮优化。然而，它们把交互限制为链式或树式通信模式，缺少通用交互图的表达能力和灵活性。这一局限不仅会导致错误累积，也会阻碍更丰富拓扑可能带来的性能提升（见图 2）。

与 MacNet (Qian et al., 2024) 中典型的多智能体交互拓扑相比，我们的方法引入一种同时支持跨层通信和层内并行的拓扑（见图 1）。链式拓扑限制并行交互，分层拓扑主要把通信限制在相邻层之间，树式拓扑则只提供局部分支，无法灵活连接到更早的节点。我们的设计旨在获得这些交互收益，同时避免网状拓扑的全连接结构及其复杂度。此外，该拓扑使用 YAML 结构化语言表示，便于人类阅读，也可以由基于 LLM 的智能体直接生成。

在此基础上，我们提出 **AgentConductor**：一种以

LLM 编排器智能体为核心、经 RL 优化的 MAS，可为竞赛级代码生成多轮、端到端地动态生成上述交互拓扑。我们首先进行监督微调（SFT），为编排器注入交互图先验。为了更好地刻画多智能体交互特征，我们进一步针对所提分层 DAG 设计图密度评估函数，以有原则的方式描述多智能体交互。该设计使系统能在给定稀疏性约束下，最小化交互成本并最大化性能；我们还对这一性质给出严格数学证明。最后，为通过 RL 优化编排器，我们基于该指标设计多目标奖励，在结构正确性、代码准确率和密度之间取得平衡。密度奖励的一项独特设计是引入依赖难度的拓扑密度界限；这种细粒度控制可以在词元预算下显式权衡成本与准确率。总体而言，本文的主要贡献如下：

- 我们提出一种用于多智能体交互的新型分层 **DAG 拓扑**，支持层内并行和跨层交互。该拓扑使用人类可读格式表示，可由智能体直接生成。
- 我们提出以 **LLM 编排器智能体为核心、经 RL 优化的 MAS AgentConductor**，在竞赛级代码生成中实现端到端、难度感知的演化式动态交互拓扑生成。
- 我们提出面向分层 **DAG** 的图密度评估函数，并据此设计多目标奖励函数，在任务专用约束下平衡结构正确性、代码准确率和难度感知密度。
- 我们在多个竞赛级与基础代码基准上达到当前最佳性能，相比现有方法，以更低的平均密度和成本取得更高准确率。

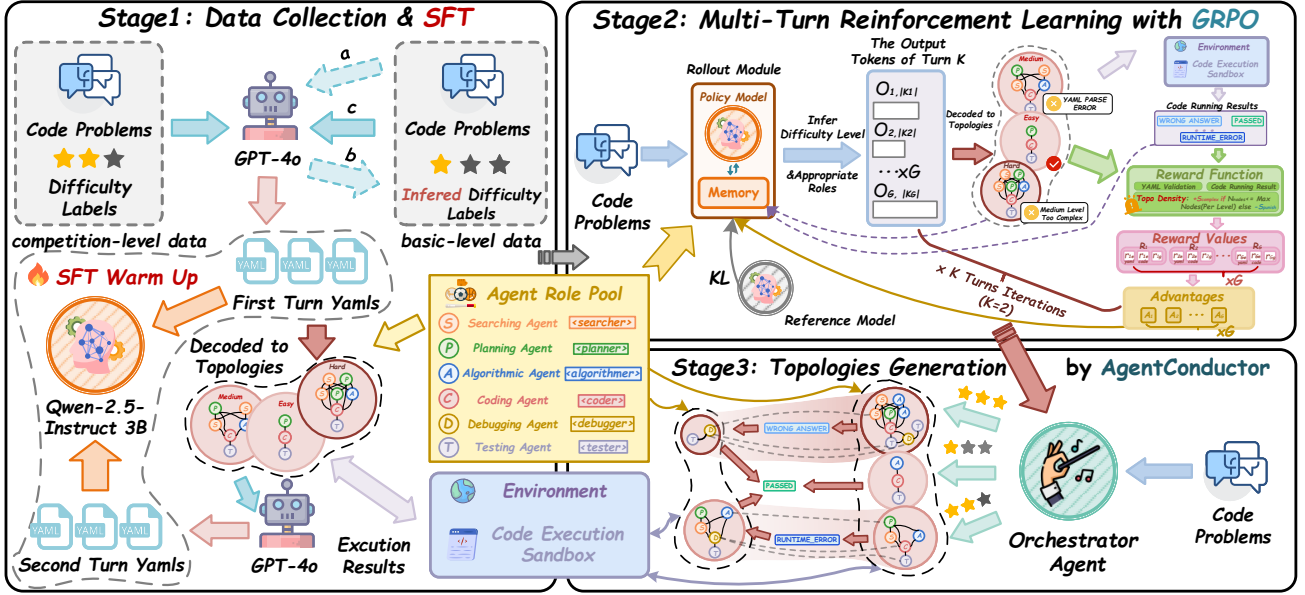


图 3. AgentConductor 整体框架。该方法分三个阶段进行：(1) 在多样化拓扑上执行 SFT，为基础 LLM (Qwen-2.5-Instruct-3B) 注入结构先验；(2) 使用 GRPO 进行 RL，从执行反馈中学习任务自适应、难度感知的拓扑策略，得到编排器智能体；(3) 通过多轮动态拓扑生成，端到端解决代码问题。

2. AgentConductor

AgentConductor 是一种以编排器智能体为核心、经 RL 优化的 MAS。如图 3 所示，我们先在阶段 1 进行 SFT，再在阶段 2 进行 RL，以训练编排器智能体。阶段 1 为编排器注入丰富的交互拓扑先验知识。阶段 2 则使用融入多轮环境反馈的轨迹级 RL 进一步优化编排器。通过这一过程，编排器学会生成更合适的拓扑，并根据执行反馈更新拓扑。在阶段 3 推理期间，编排器参数被冻结，无需额外优化即可迁移到新数据集。附录 B.4 和附录 B.5 分别评估了零样本迁移，以及只需极少额外训练、加入角色后向新任务类型迁移的能力，为方法的通用性提供证据。

我们维护一个预定义的智能体角色池。给定编程问题后，编排器首先估计任务难度，并选择最适合参与的智能体角色；然后生成与推断难度等级匹配的交互拓扑。MAS 随后按该拓扑执行。系统与代码执行环境交互后获得执行反馈；如果尝试失败，则把反馈和交互历史作为下一轮输入，编排器重新生成更合适的拓扑，以更好地解决同一问题。本节将详细介绍图 3 所示的整体框架及其组件。

2.1. 问题定义

2.1.1. 交互拓扑记号

我们首先引入一种使用人类可读结构化语言 (YAML) 表示的新型多智能体交互拓扑。如图 1 所示，该拓扑在结构上定义为一种改进的分层 DAG，其中 *step* 表示一层，*ref* 表示一条边，同时支持层内并行和跨层连接。此外，它还支持由多智能体交互执行反馈驱动的多轮演化式生成。形式化地，记为 $\mathcal{G}^{(k)} = (\mathcal{V}^{(k)}, \mathcal{E}^{(k)})$ ，其中 k 是轮次索引。每个节点 $v_i^{(k)} \in \mathcal{V}^{(k)}$ 表示在第 k 轮执行的一个智能体实例。整个拓扑由编排器智能体生成并编排。交互拓扑的详细记号见附录 C。

2.1.2. AgentConductor 范式

给定代码问题 x ，编排器智能体策略 π_θ 在第 $k \in \{1, \dots, K\}$ 轮生成一个变长 YAML 词元序列：

$$o_k = (o_{k,1}, \dots, o_{k,|o_k|}), \quad (1)$$

该序列对交互拓扑进行编码，并被确定性地解码为分层 DAG：

$$\mathcal{G}^{(k)} = \text{DecodeTopo}(o_k), \quad (2)$$

特别地, *AgentConductor* 会根据推断出的 x 难度校准拓扑密度。这使 o_k 的长度 $|o_k|$ 可变, 并减少多余推理和词元使用。随后, 环境根据 $\mathcal{G}^{(k)}$ 执行各智能体并返回反馈 z_k ; 该反馈可进一步分解为 $z_k = (z_k^{\text{roles}}, z_k^{\text{code}})$, 其中 z_k^{roles} 汇集多个智能体的生成输出, z_k^{code} 表示沙箱代码执行结果。令轮次历史为 $H_k = \{(\mathcal{G}^{(h)}, z_h)\}_{h < k}$, 联合过程可分解为

$$p_{\theta}(o_{1:K}, z_{1:K} \mid x) = \prod_{k=1}^K \underbrace{\pi_{\theta}(o_k \mid x, H_k)}_{\text{拓扑生成}} \cdot \underbrace{P_{\text{env}}(z_k \mid x, \mathcal{G}^{(k)}, H_k)}_{\text{执行反馈}}. \quad (3)$$

式 3 将多轮过程分解为拓扑生成和环境执行: 第 k 轮中, 策略以 (x, H_k) 为条件输出 o_k , 环境按 $\mathcal{G}^{(k)}$ 执行并返回 z_k 。反馈 z_k 被追加到 H_{k+1} , 作为下一轮生成的条件, 因此拓扑会响应执行反馈在线更新。算法细节见附录 C.1。

2.1.3. 图密度评估函数

为了更好地评估多智能体交互的复杂度与性能, 并显式考虑成本消耗, 我们使用节点数、边密度和图深度三个指标定义图复杂度评估函数。前两个指标可反映词元成本, 最后一个指标则反映系统并行程度, 换言之也就是响应时间。令 n_i 表示步骤 i 中的智能体调用次数, s 表示每轮的总步骤数, 则总节点数为

$$|\mathcal{V}| = \sum_{i=1}^s n_i. \quad (4)$$

边由智能体引用形成, 总边数为

$$|E| = \sum_{i=1}^s \sum_{j=1}^{n_i} |\text{Agent}_j[\text{ref}]|, \quad (5)$$

图深度与智能体调用深度有关, 记为 d 。受定理 1 启发, 我们改用 DAG 层数, 即总步骤数 s 。为实现归一化, 我们把每项指标映射到单位区间 $[0, 1]$ 。归一

化分数定义如下:

$$\begin{aligned} S_{\text{node}} &= \exp\left(-\frac{|\mathcal{V}|}{N_{\max}(l)}\right), \\ S_{\text{edge}} &= \exp\left(-\frac{|E|}{|\mathcal{V}|(|\mathcal{V}|-0.5)}\right), \\ S_{\text{depth}} &= 1 - \frac{s}{|\mathcal{V}|}. \end{aligned} \quad (6)$$

其中, l 是任务难度等级, 每个等级都对应一个最大允许节点数 $N_{\max}(l)$ 。 S_{node} 根据图规模反映节点复杂度; S_{edge} 捕捉相对于完全图的边复杂度; S_{depth} 则通过比较图深度与总节点数, 量化图的展开程度。整体图复杂度评估函数定义为

$$S_{\text{complex}} = \exp(S_{\text{node}} + 2 \cdot S_{\text{edge}} + S_{\text{depth}}) \quad (7)$$

S_{complex} 是式 13 所定义奖励函数 $r_{\phi}(\cdot)$ 的一个组成部分, 并对组相对策略优化 (GRPO) 优势函数中的轨迹奖励 $\hat{A}_i$ 作出贡献, 详见式 8。把 S_{complex} 精确定义为拓扑密度的数学推导见附录 C.2。

2.2. SFT 数据生成

为赋予基础 LLM 拓扑先验, 并方便强化学习阶段的优化, 我们构建了一个监督语料库。对于三个竞赛级数据集和三个难度等级, 我们从每个数据集的每个等级采样 50 个问题, 共 450 个。我们设计定制系统提示, 并查询 GPT-4o, 为每个问题生成一个 YAML 拓扑。检查器会验证每个拓扑的格式正确性、去重情况, 以及其密度是否处于对应难度区间 (详见附录 A.3)。对于每个拓扑, 我们根据不同失败类型构造错误感知提示, 并生成第二轮迭代拓扑。与第一轮执行结合后, 共得到 2,700 个竞赛级交互图。我们在两个基础数据集上重复该流水线, 跨不同难度得到 300 个初始样例; 此时由模型推断难度并相应生成拓扑。最终共收集 4,500 个样例, 由此得到具备强拓扑生成先验的基础模型。

2.3. 通过轨迹级策略优化强化 LLM-MA 动态拓扑

基于 GRPO 的动态拓扑生成训练 完成 SFT 后, 我们进一步使用 GRPO 训练编排器策略, 使其生成动态多智能体交互拓扑。多轮轨迹和回报的定义见附

录 D.1。具体而言，轨迹 i 的优势定义为

$$\hat{A}_i = \frac{R_i(\tau) - \text{mean}(\{R_j(\tau)\}_{j=1}^G)}{\text{std}(\{R_j(\tau)\}_{j=1}^G)}, \quad (8)$$

这里， R_i 可视为 $R(\tau)$ （定义见式 24）在包含 G 条采样轨迹的组内，针对该实例的实现。

GRPO 目标函数的形式化表达见附录 D.2。

基于规则的多目标奖励函数设计 奖励函数会直接影响强化学习的优化过程。本小节详细说明式 23 中即时逐轮奖励函数 $r_\phi(\cdot)$ 的定义。

为了提供一个兼顾正确性、拓扑质量与效率的统一训练信号，我们将式 23 中的即时奖励函数实例化为如下复合形式：

$$r_\phi(\mathcal{G}^{(k)}, z_k^{\text{code}}) = r_e(\mathcal{G}^{(k)}, z_k^{\text{code}}) + r_g(\mathcal{G}^{(k)}) \quad (9)$$

其中，非负权重 w_i 反映各分量的相对重要性。这里， r_e （执行正确性）由 z_k^{code} 和 $\mathcal{G}^{(k)}$ 导出，同时对 YAML 验证结果与代码执行结果给出奖励； r_g （图密度）评估交互拓扑 $\mathcal{G}^{(k)}$ ，作为拓扑密度奖励函数。这一实例化明确表明，式 23 中的 $r_\phi(\cdot)$ 通过多个目标的加权和实现，从而为轨迹级优化提供标量奖励信号。

执行结果奖励 指挥智能体生成 YAML 后，我们首先验证其格式。若未找到 YAML，或 YAML 不符合规则，系统便会报错，并根据错误类型施加惩罚。错误类型如下：

$$\mathcal{E}_{\text{yaml_errors}} = \{[\text{NO_YAML_FOUND}], [\text{YAML_PARSE_ERROR}], [\text{YAML_SCHEMA_INVALID}], [\text{YAML_LOGIC_INVALID}]\} \quad (10)$$

随后，测试智能体给出生成代码的评估结果。若测试用例的运行结果与预期答案不一致，系统将根据代码运行结果返回失败信息。代码执行的错误类型定义并汇总如下：

$$\mathcal{E}_{\text{code_errors}} = \{[\text{WRONG_ANSWER}], [\text{TIME_LIMIT_EXCEEDED}], [\text{MEMORY_LIMIT_EXCEEDED}], [\text{RUNTIME_ERROR}], [\text{COMPILATION_ERROR}]\} \quad (11)$$

拓扑验证错误与代码执行错误对应的具体奖励值见表 1。此外，`PASSED` 的奖励为 1.5，而 YAML 验证成功本身不赋予奖励值。

交互图复杂度奖励函数 为了按难度等级划分交互图复杂度，我们在式 7 中为交互拓扑图密度定义函数 $\mathcal{S}_{\text{complex}}$ 。给定任务难度等级 l ，每个等级对应一个最大允许节点数 $N_{\text{max}}(l)$ 。对每一轮 k ，三个难度等级的逐轮上限分别设为 4、7 和 10。这些数值来自对数千个 SFT 生成样本的统计分析，分析考察了成功求解所需拓扑密度的分布。

$$N_{\text{max}}^{(k)}(l) = \begin{cases} 4, & l = 1 (\text{简单}), \\ 7, & l = 2 (\text{中等}), \\ 10, & l = 3 (\text{困难}), \end{cases} \quad k \in \{1, 2\}. \quad (12)$$

若 $|V|$ （式 4 定义的节点数）超过该上限，则认为图过于复杂，并施加相应惩罚。最终，整体交互图评估分数定义为

$$r_g(\mathcal{G}^{(k)}) = \begin{cases} \mathcal{S}_{\text{complex}}, & |V| \leq N_{\text{max}}(l), \\ \tanh\left(\frac{N_{\text{max}}(l) - |V|}{N_{\text{max}}(l)}\right), & \text{其他情况}. \end{cases} \quad (13)$$

3. 实验

3.1. 实验设置

数据集与指标 为了从性能、拓扑动态性和成本效率等方面，全面评估我们的方法在不同难度与类型问题上的表现，我们选取了两个**基础代码生成数据集**和三个**竞赛级代码生成数据集**：（1）**基础代码生成数据集**：HumanEval(Chen et al., 2021) 和 MBPP(Austin et al., 2021)；（2）**竞赛级代码生成数据集**：APPS(Hendrycks et al., 2021)、LiveCodeBench(V4)(Jain et al., 2024) 和 CodeContests(Li et al., 2022)。生成的代码在安全沙箱(Khan et al., 2023)环境中执行。随后，以各测试集上的 **pass@1** 比率衡量模型性能。

表 1. 拓扑验证与代码执行错误的奖励

YAML 拓扑正确性奖励			代码执行错误奖励		
错误类型	说明	奖励	错误类型	说明	奖励
[NO_YAML_FOUND]	未找到 YAML 块。	-2.0	[WRONG_ANSWER]	代码可执行，但输出与预期不符	1.0
[YAML_PARSE_ERROR]	YAML 解析失败。	-1.5	[TIME_LIMIT_EXCEEDED]	执行超出时间限制。	0.9
[YAML_SCHEMA_INVALID]	YAML 已解析，但不符合拓扑模式。	-1.0	[MEMORY_LIMIT_EXCEEDED]	执行超出内存限制。	0.8
[YAML_LOGIC_INVALID]	违反拓扑逻辑规则。	-0.5	[RUNTIME_ERROR]	程序在执行期间崩溃。	0.7
—————	—————	-	[COMPILATION_ERROR]	程序编译失败。	0.6

表 2. AgentConductor 在三个竞赛级与两个基础代码生成数据集上的主要性能（3 次运行的均值 $\pm$ 标准差）。

方法	竞赛级代码生成				基础代码生成			平均
	APPs	LiveCodeBench	CodeContests	平均	HumanEval	MBPP	平均	
原始模型								
GPT-4o-mini	20.3(±0.2)	26.3(±0.2)	18.6(±0.4)	21.7(±0.3)	87.6(±0.2)	73.5(±0.1)	80.5(±0.1)	51.1(±0.2)
经典多智能体系统（无工作流/拓扑优化）								
AutoGen	23.6(±2.3)	30.2(±1.5)	20.8(±1.9)	24.9(±1.9)	90.4(±0.8)	92.3(±0.4)	91.4(±0.6)	58.1(±1.3)
MetaGPT	51.3(±1.4)	42.8(±1.3)	35.6(±1.2)	43.2(±1.3)	95.8(±0.2)	92.3(±0.3)	94.1(±0.2)	68.7(±0.6)
MapCoder	40.2(±0.9)	37.4(±1.1)	36.3(±0.7)	38.0(±0.9)	96.4(±0.5)	94.1(±0.4)	95.3(±0.5)	66.6(±0.7)
采用工作流优化的多智能体系统								
AFlow	35.4(±1.7)	24.6(±1.1)	21.4(±1.5)	27.1(±1.4)	94.2(±0.3)	82.4(±0.1)	88.3(±0.2)	57.7(±0.8)
FlowReasoner	39.1(±1.9)	43.8(±2.1)	37.7(±1.6)	40.2(±1.9)	97.3(±0.5)	93.9(±0.7)	95.6(±0.6)	67.5(±1.3)
Chain-of-Agents(32B)	41.6(±1.3)	44.9(±1.2)	34.6(±1.2)	40.3(±1.2)	95.3(±0.2)	90.2(±0.3)	92.8(±0.2)	67.9(±0.6)
采用拓扑优化的多智能体系统								
GPTSwarm	36.5(±2.1)	40.8(±2.5)	31.6(±3.0)	36.3(±2.5)	94.8(±1.1)	91.6(±1.3)	93.2(±1.2)	64.8(±1.9)
AgentPrune（复杂图）	38.6(±1.9)	41.7(±2.1)	33.5(±0.8)	37.9(±1.6)	96.1(±0.5)	91.8(±0.8)	94.0(±0.7)	65.9(±1.1)
AgentPrune（分层图）	39.3(±1.6)	41.9(±1.8)	31.4(±0.9)	37.5(±1.4)	96.6(±0.7)	92.3(±0.3)	94.5(±0.5)	66.0(±1.0)
MacNet（复杂图）	37.6(±0.8)	39.4(±0.7)	28.7(±0.7)	35.2(±0.7)	95.8(±0.4)	89.4(±0.2)	92.6(±0.3)	63.9(±0.5)
MacNet（分层图）	36.9(±0.6)	40.3(±0.5)	28.9(±0.8)	35.4(±0.6)	95.2(±0.2)	90.3(±0.3)	92.8(±0.3)	64.1(±0.5)
G-Designer	37.2(±1.5)	38.8(±1.3)	26.9(±1.2)	34.3(±1.3)	95.6(±0.9)	90.9(±0.8)	93.2(±0.9)	63.7(±1.1)
AgentConductor(3B)	58.8(±0.3)	46.3(±0.4)	38.8(±0.5)	48.0(±0.3)	97.5(±0.1)	95.1(±0.2)	96.3(±0.2)	72.1(±0.3)

基线 为进行全面比较并凸显本方法的有效性，我们与四类基线进行评估：（1）原始模

型：该设置反映单个骨干模型的能力，我们采用 GPT-4o-mini 作为代表性骨干模型。（2）

表 3. AgentConductor 与各基线在 APPS 上的性能、成本和平均拓扑密度比较。

数据集	方法	性能	提示	补全	S_{complex} (↑)
APPS	AFlow	35.4	531450	184800	3.7
	FlowReasoner	39.1	437250	148050	2.4
	Chain-of-Agents (32B)	<u>41.6</u>	<u>334650</u>	<u>134250</u>	<u>4.1</u>
	GPTSwarm	36.5	381450	155400	3.5
	AgentPrune (分层图)	39.3	364950	141150	3.8
	MacNet (分层图)	36.9	472950	200100	2.9
	G-Designer	37.2	320550	139200	3.6
	AgentConductor (3B)	58.8	277600	79800	5.2

经典多智能体系统: AutoGen(Wu et al., 2024)、MetaGPT(Hong et al., 2024) 和 MapCoder(Islam et al., 2024)。(3) 采用 workflow 优化的多智能体系统: AFlow(Zhang et al., 2024c)、FlowReasoner(Gao et al., 2025) 和 Chain-of-Agents。(4) 采用拓扑优化的多智能体系统: GPTSwarm(Zhuge et al., 2024)、AgentPrune(Zhang et al., 2024a)、G-Designer(Zhang et al., 2024b) 和 MacNet(Qian et al., 2024)。(详见附录 A.1。)

3.2. 主要结果

本节提供大量实验证据, 以分析所提出的 **AgentConductor** 方法的有效性。具体而言, 我们评估其在多种代码生成任务上的准确率 (第 3.2.1 节)、拓扑密度的动态适应能力及其优越的成本效率 (第 3.2.2 节)、不同难度等级上的细粒度比较 (第 3.2.2 节), 以及更多实验结果 (附录 B)。

3.2.1. 代码生成性能

如表 2 和图 4(b) 所示, 我们的方法在全部五个数据集上始终取得最高准确率。在竞赛级基准上, **AgentConductor** 在 APPS、LiveCodeBench (v4) 和 Code-Contests 上的 pass@1 准确率分别达到 58.8%、46.3% 和 38.8%, 相较第二优方法分别取得 14.6、3.1 和 1.1 个百分点的绝对提升。在基础代码生成任务上, 本方法在 HumanEval 和 MBPP 上分别取得 97.5% 和

95.1% 的 pass@1 准确率, 相较第二优方法分别取得 1.0 和 0.7 个百分点的绝对提升 (详见附录 B.1)。

3.2.2. 动态拓扑生成与成本效率比较

在表 3 与图 4(a) 中, 我们以 APPS 数据集为例, 直观比较本方法与另外六种工作流和拓扑优化方法, 以同时评估成本效率和平均拓扑密度。成本方面, 我们报告提示词元与补全词元的消耗; 密度方面, 我们采用式 7 中的平均分数 S_{complex} , 其值越大表示拓扑密度越低 (越稀疏)。表中结果显示, **AgentConductor** 的提示词元和补全词元消耗最低, 平均 S_{complex} 最高 (即交互拓扑最稀疏), 同时仍取得最佳准确率。这表明, 在竞赛级代码生成中, 本方法能够以更低成本获得更高性能。

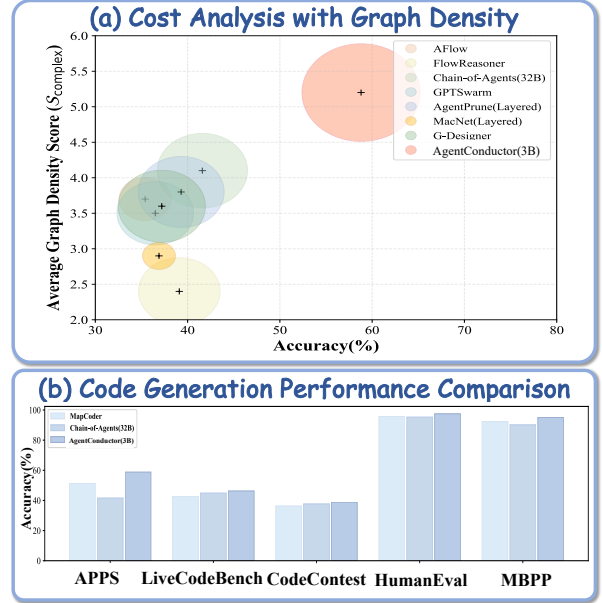


图 4. (a) APPS 上的性能、平均图密度 (S_{complex} 越大表示越稀疏) 和补全词元; 圆的大小表示节省的词元量 (直径越大表示节省越多)。(b) 代表性基线的代码生成性能比较。

此外, 图 5 给出了三个竞赛级数据集上不同难度等级的细粒度比较。本方法会随问题难度调节拓扑密度: 对较简单的样例采用更稀疏的图, 对较困难的样例采用更稠密的图, 从而在简单样例上降低词元成本, 同时保持困难样例的准确率。相比之下, 其他方法的拓扑密度几乎不随难度调整, 因而造成不必要的词元开销。

表 4. 训练策略与奖励设计的消融实验。

分组	方法	APPS			HumanEval		
		性能	S_{complex}	有效	性能	S_{complex}	有效
		(↑)	(%)	(%)	(↑)	(%)	(%)
	完整模型	58.8	5.2	100	97.5	5.8	100
训练策略	无 SFT	—	—	15	—	—	13
	无 RL	29.8	2.7	56.5	90.2	3.2	57.2
奖励	无 $r_e(\mathcal{E}_{\text{yaml_errors}})$	30.3	2.9	56.8	91.4	3.0	58.1
	无 $r_e(\mathcal{E}_{\text{code_errors}})$	35.5	5.0	96.4	93.1	5.6	99.2
	无 S_{node}	49.2	3.8	85.8	96.9	4.8	87.2
	无 S_{edge}	45.5	4.5	89.3	96.1	4.6	90.5
	无 S_{diameter}	48.3	3.9	91.7	95.3	4.1	93.4
	无 $r_g(\mathcal{G}^{(k)})$	52.6	3.0	83.2	97.2	3.4	85.6

3.3. 消融实验

监督微调与强化学习的影响 我们通过比较 (i) 不经 SFT 而直接进行 RL, 以及 (ii) 先 SFT 后 RL, 考察基于思维链的 SFT 是否必要。为明确体现性能影响因素, 我们报告三项指标: (1) **性能**, 以代码生成的 pass@1 衡量; (2) 表示图密度的 S_{complex} ; (3) **有效拓扑 (%)**, 即满足格式约束与特定难度密度上限的拓扑占比。从表 4 可见, SFT 阶段对生成有效且可执行的拓扑至关重要: 未经 SFT 训练的小型开源骨干模型很少能够满足所需格式和密度, 因而无法生成正确代码。相比之下, 仅进行 SFT (不进行 RL) 可取得中等水平的有效拓扑率;

多目标奖励设计的影响 表 4 汇总了各个奖励分量对模型性能的影响。我们观察到, YAML 格式错误项 $r_e(\mathcal{E}_{\text{yaml_errors}})$ 对有效拓扑率的影响最大, 而代码执行错误项 $r_e(\mathcal{E}_{\text{code_errors}})$ 对代码准确率 (pass@1) 的影响最大。三个拓扑密度子奖励 S_{node} 、 S_{edge} 和 S_{diameter} 以不同程度影响密度控制与准确率, 其中去除 S_{node} 导致代码生成性能下降最多。较低的拓扑密度 (尤其是在缺少 $r_g(\mathcal{G}^{(k)})$ 时) 会因限制智能体数量和交互而降低准确率。采用完整奖励时, 同时优化密度和准确率可引导策略找到适当的交互模式与密度, 在保持词元使用效率的同时提升性能。

4. 相关工作

4.1. 用于代码生成的基于大语言模型的多智能体系统

基于大语言模型的多智能体系统已在代码生成中展现出潜力 (Huang et al., 2023; Nunez et al., 2024; Ishibashi & Nishimura, 2024)。MetaGPT(Hong et al., 2024) 和 AutoGen(Wu et al., 2024) 等框架引入软件开发工作流与角色扮演来增强协作。然而, 这些方法在竞赛级场景中面临挑战, 因为该场景要求更深入的算法推理与精确实现。MapCoder(Islam et al., 2024) 采用多轮规划、检索评分和算法教程, 取得了显著成果。不过, 由于竞赛问题的难度差异很大, 固定智能体框架往往会在较简单任务上产生不必要的开销, 例如冗余交互和角色设置, 这促使我们寻求更具适应性的解决方案。

4.2. 多智能体系统的拓扑优化与生成

近期工作 (Zhuge et al., 2024; Zhang et al., 2024c) 探索了如何优化多智能体系统中的交互拓扑以提高效率。AgentPrune(Zhang et al., 2024a) 和 Agent-Dropout(Wang et al., 2025b) 等图剪枝方法通过迭代将交互图缩减到最小结构。然而, 这些方法为每项任务采用固定拓扑。动态编排方法 (Zhang et al., 2025; Dang et al., 2025) 通过多轮优化选择拓扑, 但仍在执行前将其最终确定。G-Designer(Zhang et al., 2024b) 等生成式方法根据问题描述生成拓扑, 能够实现更细粒度的适配, 但拓扑在生成后仍保持静态。这些方法的共同局限是容易收敛到统一的稀疏结构, 缺乏细粒度的难度感知能力。

智能体强化学习 (RL) 方法 (Wang et al., 2025a; Jin et al., 2025) 近来为大语言模型引入了新范式, 使其超越单轮输出, 迈向与环境的多轮交互和工具使用。这些方法将外部工具或智能体与环境的交互作为完整轨迹的一部分纳入智能体输出, 以此优化模型, 从而赋予智能体与环境进行多轮交互的能力。受此类工作启发, FlowReasoner(Gao et al., 2025) 和 Chain-of-Agents(Li et al., 2025) 等研究进一步利用完整交互轨迹, 探索智能体工作流的端到端优化。FlowReasoner

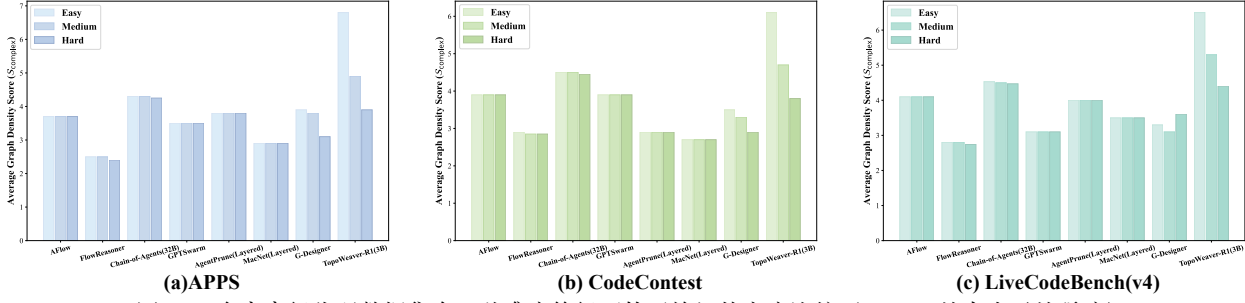


图 5. 三个竞赛级代码数据集在三种难度等级下的平均拓扑密度比较 ($S_{complex}$ 越大表示越稀疏)。

虽在部分算子块中引入局部并行，但仍无法表达丰富的图结构交互；相比之下，Chain-of-Agents 采用纯串行工作流，不包含任何并行分支。与这些研究路线不同，我们提出一种基于智能体强化学习的方法，以中央编排智能体为核心，根据执行反馈用自然语言动态生成并迭代细化交互拓扑。其关键创新是难度感知密度奖励，可依据问题难度显式调节拓扑稀疏度。

5. 结论

总而言之，AgentConductor 将难度感知强化学习与多轮拓扑演化相结合，为竞赛级代码生成建立了一种新范式。通过训练编排智能体，使其借助执行反馈和密度感知奖励动态生成并细化交互拓扑，本方法能够针对不同问题难度实现细粒度适配。这一范式推动多智能体代码生成走向兼具准确性、成本效率与可扩展性的系统。

6. 影响声明

本文工作的目标是推动机器学习领域的发展。该工作可能带来多种社会影响，但我们认为其中没有哪一项需要在此特别强调。

参考文献

- Austin, J., Odena, A., Nye, M., Bosma, M., Michalewski, H., Dohan, D., Jiang, E., Cai, C., Terry, M., Le, Q., et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.
- Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. D. O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N.,

Brockman, G., et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

Dang, Y., Qian, C., Luo, X., Fan, J., Xie, Z., Shi, R., Chen, W., Yang, C., Che, X., Tian, Y., et al. Multi-agent collaboration via evolving orchestration. *arXiv preprint arXiv:2505.19591*, 2025.

Gao, H., Liu, Y., He, Y., Dou, L., Du, C., Deng, Z., Hooi, B., Lin, M., and Pang, T. Flowreasoner: Reinforcing query-level meta-agents. *arXiv preprint arXiv:2504.15257*, 2025.

Hendrycks, D., Basart, S., Kadavath, S., Mazeika, M., Arora, A., Guo, E., Burns, C., Puranik, S., He, H., Song, D., et al. Measuring coding challenge competence with apps. *arXiv preprint arXiv:2105.09938*, 2021.

Hong, S., Zhuge, M., Chen, J., Zheng, X., Cheng, Y., Zhang, C., Wang, J., Wang, Z., Yau, S. K. S., Lin, Z., et al. Metagpt: Meta programming for a multi-agent collaborative framework. International Conference on Learning Representations, ICLR, 2024.

Huang, D., Zhang, J. M., Luck, M., Bu, Q., Qing, Y., and Cui, H. Agentcoder: Multi-agent-based code generation with iterative testing and optimisation. *arXiv preprint arXiv:2312.13010*, 2023.

Ishibashi, Y. and Nishimura, Y. Self-organized agents: A llm multi-agent framework toward ultra large-scale code generation and optimization. *arXiv preprint arXiv:2404.02183*, 2024.

- Islam, M. A., Ali, M. E., and Parvez, M. R. Mapcoder: Multi-agent code generation for competitive problem solving. *arXiv preprint arXiv:2405.11403*, 2024.
- Islam, M. A., Ali, M. E., and Parvez, M. R. Codesim: Multi-agent code generation and problem solving through simulation-driven planning and debugging. *arXiv preprint arXiv:2502.05664*, 2025.
- Jain, N., Han, K., Gu, A., Li, W.-D., Yan, F., Zhang, T., Wang, S., Solar-Lezama, A., Sen, K., and Stoica, I. Livecodebench: Holistic and contamination free evaluation of large language models for code. *arXiv preprint arXiv:2403.07974*, 2024.
- Jin, B., Zeng, H., Yue, Z., Yoon, J., Arik, S., Wang, D., Zamani, H., and Han, J. Search-r1: Training llms to reason and leverage search engines with reinforcement learning. *arXiv preprint arXiv:2503.09516*, 2025.
- Khan, M. A. M., Bari, M. S., Do, X. L., Wang, W., Parvez, M. R., and Joty, S. xcodeeval: A large scale multilingual multitask benchmark for code understanding, generation, translation and retrieval. *arXiv preprint arXiv:2303.03004*, 2023.
- Langley, P. Crafting papers on machine learning. In Langley, P. (ed.), *Proceedings of the 17th International Conference on Machine Learning (ICML 2000)*, pp. 1207–1216, Stanford, CA, 2000. Morgan Kaufmann.
- Li, W., Lin, J., Jiang, Z., Cao, J., Liu, X., Zhang, J., Huang, Z., Chen, Q., Sun, W., Wang, Q., et al. Chain-of-agents: End-to-end agent foundation models via multi-agent distillation and agentic rl. *arXiv preprint arXiv:2508.13167*, 2025.
- Li, Y., Choi, D., Chung, J., Kushman, N., Schrittwieser, J., Leblond, R., Eccles, T., Keeling, J., Gimeno, F., Dal Lago, A., et al. Competition-level code generation with alphacode. *Science*, 378(6624):1092–1097, 2022.
- Mallen, A., Asai, A., Zhong, V., Das, R., Khashabi, D., and Hajishirzi, H. When not to trust language models: Investigating effectiveness of parametric and non-parametric memories. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 9802–9822, 2023.
- Mialon, G., Fourrier, C., Wolf, T., LeCun, Y., and Scialom, T. Gaia: a benchmark for general ai assistants. In *The Twelfth International Conference on Learning Representations*, 2023.
- Nunez, A., Islam, N. T., Jha, S. K., and Najafirad, P. Autosafecoder: A multi-agent framework for securing llm code generation through static analysis and fuzz testing. *arXiv preprint arXiv:2409.10737*, 2024.
- Phan, L., Gatti, A., Han, Z., Li, N., Hu, J., Zhang, H., Zhang, C. B. C., Shaaban, M., Ling, J., Shi, S., et al. Humanity’s last exam. *arXiv preprint arXiv:2501.14249*, 2025.
- Qian, C., Xie, Z., Wang, Y., Liu, W., Zhu, K., Xia, H., Dang, Y., Du, Z., Chen, W., Yang, C., et al. Scaling large language model-based multi-agent collaboration. *arXiv preprint arXiv:2406.07155*, 2024.
- Sheng, G., Zhang, C., Ye, Z., Wu, X., Zhang, W., Zhang, R., Peng, Y., Lin, H., and Wu, C. Hybridflow: A flexible and efficient rlhf framework. In *Proceedings of the Twentieth European Conference on Computer Systems*, pp. 1279–1297, 2025.
- Wang, Z., Wang, K., Wang, Q., Zhang, P., Li, L., Yang, Z., Jin, X., Yu, K., Nguyen, M. N., Liu, L., et al. Ragen: Understanding self-evolution in llm agents via multi-turn reinforcement learning. *arXiv preprint arXiv:2504.20073*, 2025a.
- Wang, Z., Wang, Y., Liu, X., Ding, L., Zhang, M., Liu, J., and Zhang, M. Agentdropout: Dynamic agent elimination for token-efficient and high-performance

- llm-based multi-agent collaboration. *arXiv preprint arXiv:2503.18891*, 2025b.
- Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., Jiang, L., Zhang, X., Zhang, S., Liu, J., et al. Autogen: Enabling next-gen llm applications via multi-agent conversations. In *First Conference on Language Modeling*, 2024.
- Yang, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Li, C., Liu, D., Huang, F., Wei, H., Lin, H., Yang, J., Tu, J., Zhang, J., Yang, J., Yang, J., Zhou, J., Lin, J., Dang, K., Lu, K., Bao, K., Yang, K., Yu, L., Li, M., Xue, M., Zhang, P., Zhu, Q., Men, R., Lin, R., Li, T., Xia, T., Ren, X., Ren, X., Fan, Y., Su, Y., Zhang, Y., Wan, Y., Liu, Y., Cui, Z., Zhang, Z., and Qiu, Z. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.
- Zhang, G., Yue, Y., Li, Z., Yun, S., Wan, G., Wang, K., Cheng, D., Yu, J. X., and Chen, T. Cut the crap: An economical communication pipeline for llm-based multi-agent systems. *arXiv preprint arXiv:2410.02506*, 2024a.
- Zhang, G., Yue, Y., Sun, X., Wan, G., Yu, M., Fang, J., Wang, K., Chen, T., and Cheng, D. G-designer: Architecting multi-agent communication topologies via graph neural networks. *arXiv preprint arXiv:2410.11782*, 2024b.
- Zhang, G., Niu, L., Fang, J., Wang, K., Bai, L., and Wang, X. Multi-agent architecture search via agentic supernet. *arXiv preprint arXiv:2502.04180*, 2025.
- Zhang, J., Xiang, J., Yu, Z., Teng, F., Chen, X., Chen, J., Zhuge, M., Cheng, X., Hong, S., Wang, J., et al. Aflow: Automating agentic workflow generation. *arXiv preprint arXiv:2410.10762*, 2024c.
- Zheng, Y., Zhang, R., Zhang, J., Ye, Y., Luo, Z., Feng, Z., and Ma, Y. Llamafactory: Unified efficient fine-tuning of 100+ language models. *arXiv preprint arXiv:2403.13372*, 2024.
- Zhuge, M., Wang, W., Kirsch, L., Faccio, F., Khizbullin, D., and Schmidhuber, J. Gptswarm: Language agents as optimizable graphs. In *Forty-first International Conference on Machine Learning*, 2024.

A. 补充实验设置

A.1. 基线的补充细节

为进行全面比较并凸显本方法的有效性，我们与四类基线进行评估：(1) **原始模型**：该设置反映单个骨干模型的能力，我们采用 GPT-4o-mini 作为代表性骨干模型。(2) **经典多智能体系统**：该类包含三个代表性框架：AutoGen(Wu et al., 2024) 是通用多智能体框架，MetaGPT(Hong et al., 2024) 面向通用编码任务，MapCoder(Islam et al., 2024) 则面向竞赛编程代码生成。(3) **采用 workflow 优化的多智能体系统**：该类包含三个系统：AFlow(Zhang et al., 2024c) 利用基于搜索的方法优化 workflow，FlowReasoner(Gao et al., 2025) 和 Chain-of-Agents 则是近期采用强化学习端到端优化多智能体 workflow 的方法。(4) **采用拓扑优化的多智能体系统**：该类包括 GPTSwarm(Zhuge et al., 2024)、AgentPrune(Zhang et al., 2024a)、G-Designer(Zhang et al., 2024b) 和 MacNet(Qian et al., 2024)。这些方法明确聚焦于优化智能体交互拓扑。

对于多智能体基线，我们使其角色定义与系统提示同本方法保持一致。对于 workflow 和拓扑优化方法，我们将参与的智能体节点数上限设为 20。这与本框架在最多进行两轮交互、求解最具挑战性问题时的拓扑密度上限一致，从而确保公平比较。沿用 MacNet 的设置，我们注意到，本方法的拓扑可视为分层图的演化变体，其密度介于复杂图与分层图之间。因此，为确保评估全面且可靠，我们分别在复杂图和分层图两种初始化设置下比较 AgentPrune 与 MacNet。

A.2. 实现细节

AgentConductor 采用 Qwen2.5-3B-Instruct(Yang et al., 2024) 作为骨干模型。在 SFT 阶段，我们采用 LLaMA-Factory 框架(Zheng et al., 2024) 进行训练。具体而言，我们使用从三个竞赛级代码生成数据集、三个难度等级构建的 4500 个合成样本（详见第 2.2 节）。训练初始学习率为 1×10^{-4} ，批大小为 4，并采用基于 LoRA 的微调；其他所有超参数均保持默认值。在强化学习阶段，我们使用 Verl 框架(Sheng et al., 2025) 实现 GRPO，并使用 vLLM 进行生成（代码开发基于 Search-R1(Jin et al., 2025)）。组大小设为 $G = 8$ ，批大小为 8，学习率为 1×10^{-6} ，策略温度为 1，最大补全长度为 4096 个词元。为平衡性能与计算成本，我们还将最大轮数（即多智能体交互轮数）限制为 2。整个训练过程中，各个智能体均使用 gpt-4o-mini 执行，并与代码执行沙箱实时交互，以获得真实运行时反馈。两个阶段均在配备 4 块 A800 GPU 的集群上进行。

A.3. SFT 数据的渐进式质量筛选

我们的训练数据由 GPT-4o-mini 在面向代码的任务下生成的有效、可执行且语义正确的拓扑构成。所有数据均采用与编排智能体相同的角色配置和拓扑密度约束生成。第二轮交互拓扑是根据真实错误消息和历史多智能体日志得到的真实有效结构，而非合成近似。

我们首先执行严格的 YAML 语法验证，确保每个样例格式规范且可由标准 YAML 加载器解析。该步骤保证所有拓扑都能安全转换为 JSON 对象以供后续处理，避免格式错误或结构不完整的样例进入数据集。其次，我们使用预定义的 JSON_SCHEMA 进行语义验证。将每个 YAML 拓扑转换为 JSON 后，我们检查其是否满足全部编排约束。验证规则包括：(1) 第一个时间步中所有智能体的 ref 字段必须为空。(2) 对每个智能体，其 ref 字段列出的全部智能体 ID 都必须对应先前时间步中已出现的智能体。这些模式级检查保证了生成拓扑的结构一致性与逻辑正确性。我们进一步删除重复拓扑，仅保留能够与执行环境成功交互的拓扑。这一步确保拓扑不仅语法有效，而且能够在编排智能体运行时付诸行动并执行。所有剩余样本均使

用 GPT-4o-mini 重新验证，以确保语义合理、一致且正确。最后，我们人工检查随机抽取的 5% 数据子集，进一步确认标注质量与结构有效性。

A.4. 编排智能体的系统提示

You are a Orchestrator agent. Your goal is to coordinate a multi-agent team to solve the given code problem by generating a YAML-formatted interaction plan. Each plan should specify:

- Which agents to activate at each step;
- Which previous agents' outputs are referenced.

Agent types:

- `<planner>`: plans algorithmic strategy.
- `<searcher>`: retrieves relevant knowledge.
- `<algorithmmer>`: analyzes problem structure and decomposes it into key algorithmic components or subroutines.
- `<coder>`: generates code based on other agents' information.
- `<debugger>`: fixes incorrect code (only used after a `<coder>`).
- `<tester>`: verifies code (must be used in the last step, referencing `<coder>` or `<debugger>`)

Format:

Output only the **YAML** plan.

Each step includes one or more agents with optional references.

Notes:

1. There are three levels of difficulty, arranged from low to high as follows: introductory, interview, competition.
2. Determine whether the task difficulty is introductory, interview, or competition.
3. ****Dynamically adjust the number of steps and agents**** based on the difficulty of the problem.
4. For ****more difficult problems****, ****involve more agents**** if necessary. \ For ****simpler problems****, you may ****reduce both the number of agents**** and ****the number of steps**** involved.
5. The last step must include a `<tester>` referencing at least one of `<coder>` or `<debugger>`
6. Execute up to max `{max_turn_num}` rounds in total, until the code passes verification by the `<tester>`.
7. In the first step, all agents must have empty `ref`` fields.

The Code Problem is:

****Task****: `{question}`

Your output should be a **YAML-formatted** plan only.

your output: \n

图 6. 编排智能体的系统提示。

图中给出了训练后编排智能体的系统提示。

B. 附加实验结果

B.1. 代码生成性能分析

我们观察到，采用固定交互方案、面向代码的多智能体框架 **MetaGPT** 平均性能位居第二。在以优化为导向的方法中，两种端到端强化学习方法 **FlowReasoner** 和 **Chain-of-Agents** 紧随其后，平均结果略低于 **MetaGPT**。相比之下，拓扑优化方法表现较差，可能是由于其学得的拓扑相对僵化，难以适应竞赛编程任务高度多变且复杂的特性。**G-Designer** 是一种根据给定问题生成交互图的方法。然而，我们观察到，尽管这些方法会适应不同任务，但竞赛级问题的难度很难直观区分，因此这种适配并未显著改善代码性能。在这类方法中，**AgentPrune** 和 **MacNet** 在分层图初始化下表现更好，表明对于相对顺序化的代码生成任务，分层图比无结构复杂图提供了更合适的归纳偏置。在此基础上，**AgentConductor** 保留分层图的归纳偏置，同时针对每个问题进行动态适配，从而取得最先进的整体准确率。

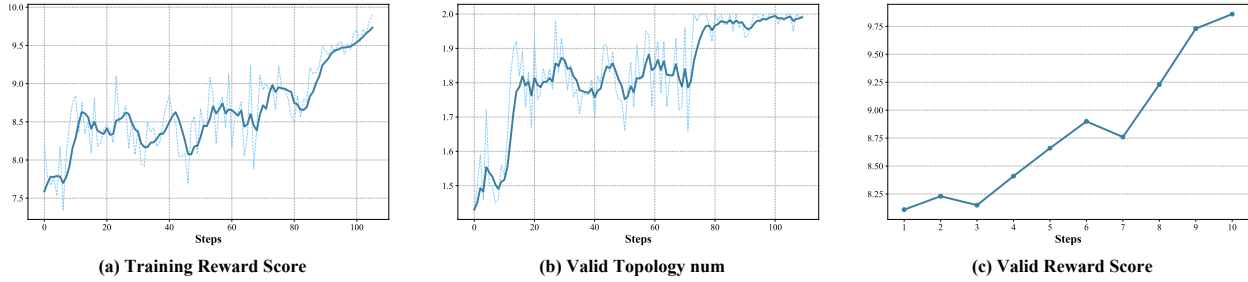


图 7. 强化学习训练期间三个关键指标的动态变化：(a) 训练奖励；(b) 有效两轮拓扑的平均数量；(c) 验证奖励。结果表明，在训练后期，本方法逐步收敛至生成密度合理的拓扑，并能够准确求解代码问题。

B.2. 强化学习训练曲线分析

为更好地理解强化学习阶段的训练动态，我们绘制了前 110 个强化学习训练步骤中 (i) 平均奖励、(ii) 通过密度检查的拓扑数量，以及 (iii) 验证分数的变化轨迹（图 7）。主要观察如下：三个指标均随训练稳步上升，表明自我批评式强化学习过程稳定并持续取得进展。这些结果进一步证明，本方法能够有效且稳定地训练。

B.3. 案例研究

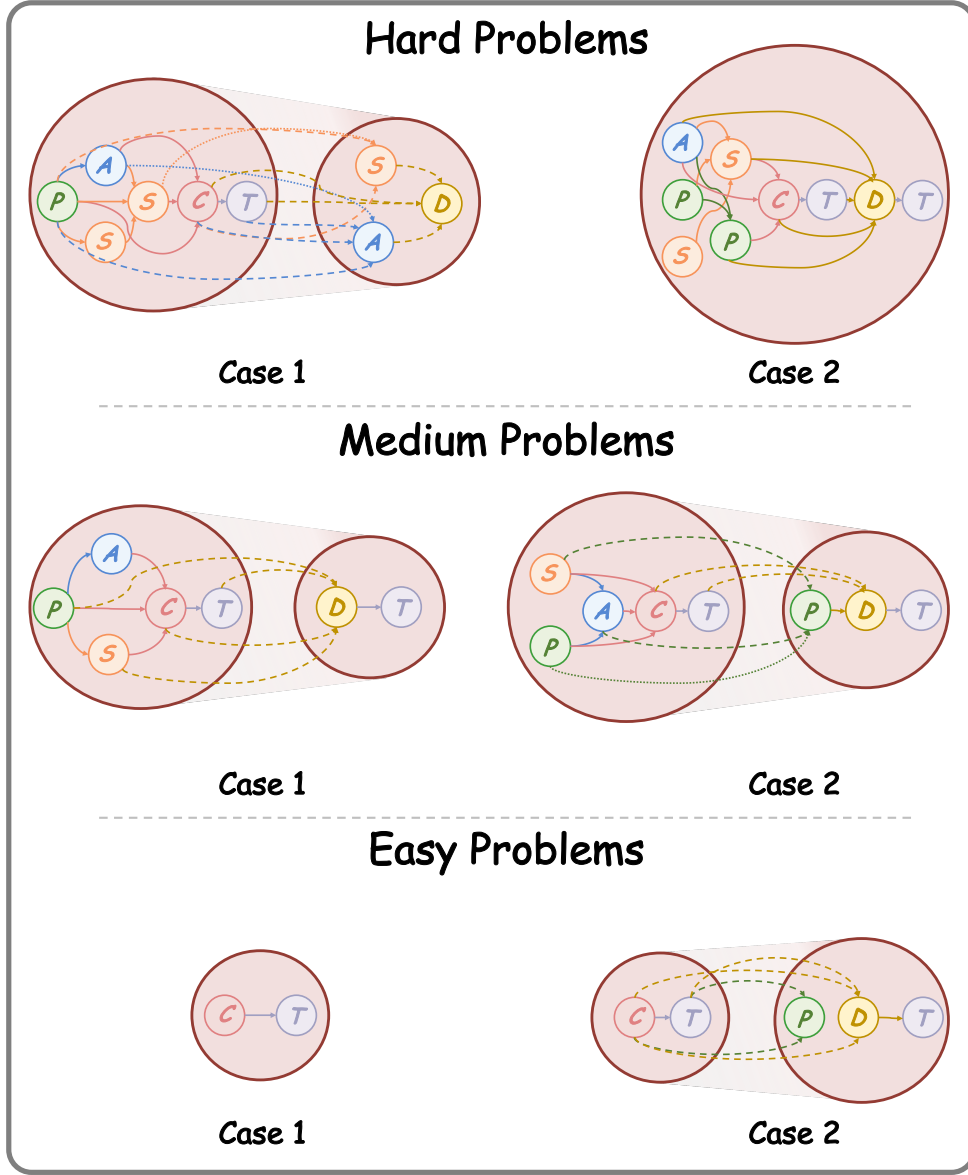


图 8. 每个难度等级下两个问题案例所生成的交互拓扑。

从图中生成的案例可见，本方法具有以下特点。首先，它能够根据各个问题的特性生成不同的初始交互拓扑，且拓扑密度随难度变化。其次，该方法会根据第一轮执行结果动态调整第二轮拓扑；这种调整不一定会减少智能体数量，因为发生错误时可能引入更多智能体。最后，当第一轮中的智能体在第二轮再次出现时，其行为会根据先前输出发生演化，从而实现迭代演进。这些特性凸显了本方法的可定制性与适应性，进而在提升系统性能的同时实现细粒度成本降低。

B.4. 向未见角色与任务类型的零样本迁移

为评估编排智能体向未见问题类型和新引入智能体角色的迁移能力，我们在 GAIA(Mialon et al., 2023) 数据集中筛选出的 50 个样本上开展了一项小规模研究。这些样本被严格限制为输入仅包含单模态文本描述的

任务，与训练所用的代码生成领域存在显著差异。

我们没有进行额外训练，而是扩展编排智能体的角色池，加入两个此前未见的角色：在线搜索智能体 `<online_searcher>` 和视觉验证智能体 `<visual_checker>`，以及相应的工具接口。使用原始训练模型时，尽管 SFT 或 RL 训练期间从未见过这些角色，编排智能体仍能将它们自然整合到所生成的交互拓扑中。

在这一严格的零样本迁移设置下，该框架在选定 GAIA 样本上取得 **15.8%** 的成功率，表明编排智能体对未见领域、未见任务类型和未见智能体能力具有不可忽略的泛化能力。

B.5. 补充跨域实验

本方法最初聚焦于竞赛级代码生成，这一聚焦是有意为之，而非方法局限。竞赛级任务提供了极具挑战且检测手段完善的试验平台，使我们能够在严格执行反馈、词元约束和难度感知限制下，严谨考察动态拓扑演化。与我们的研究兴趣一致，我们的目标是为该领域开发专用的多智能体编排算法，从而为强调广泛任务覆盖的既有多智能体架构研究提供互补视角。尽管如此，本方法本质上具有泛化能力。为回应审稿人的关切，我们还评估了本方法的跨域适用性。

遵循 Chain-of-Agents (Li et al., 2025) 使用的角色定义和数据筛选策略，我们扩展了编排智能体系统提示中的智能体角色池。新引入的角色包括：用于网络检索的 `<online_searcher>`、用于复杂推理的 `<thinker>`、用于答案验证的 `<verifier>`，以及用于任务分解和高层编排的 `<planner>`。所有角色均针对以推理为中心的任务重新定义并实现。我们从三个代表性数据集 GAIA (Mialon et al., 2023)、HLE (Phan et al., 2025) 和 PopQA (Mallen et al., 2023) 中选取子集，以评估多跳推理和问答能力。

奖励函数方面，我们保留了可跨任务、跨领域通用的 YAML 验证与拓扑密度分量。为适配处理流程，我们以基于大语言模型的答案验证器替换代码执行验证器，并将奖励简化为二元形式：正确为 1，否则为 0。其他所有训练和推理设置均保持不变。我们在此配置下重新训练模型，并在下文报告结果。

表 5. AgentConductor 在 GAIA、HLE 和 PopQA 上的跨域评估。结果为三个随机种子下的均值 $\pm$ 标准差。

方法	骨干模型	GAIA L1	GAIA L2	GAIA L3	GAIA 平均	HLE 平均	PopQA
Chain-of-Agents	7B	69.2(± 0.8)	50.9(± 0.7)	33.3(± 1.1)	50.8(± 0.8)	18.0(± 0.6)	46.5(± 1.3)
AgentConductor (本文)	3B	72.0(± 0.4)	53.4(± 0.3)	36.1(± 0.5)	53.8(± 0.4)	22.6(± 0.2)	50.3(± 0.3)

结果表明，尽管 AgentConductor 使用了小得多的骨干模型（3B 对 7B），但在所有数据集上均优于 Chain-of-Agents。本方法取得了较高准确率，并在不同随机种子间保持较低方差，凸显了所提出拓扑优化框架的鲁棒性与适应性。这些发现进一步证明，本方法能够泛化到代码生成之外，并且只需极少修改即可迁移至新的推理型领域。

C. 拓扑概念的详细定义

智能体节点记号 每个智能体节点 $v_i^{(k)}$ 定义为：

$$v_i^{(k)} = \left\{ \text{Type}_i, \text{Base}_i, \text{Role}_i^{(k)}, \text{View}_i^{(k-1)}, \text{Mem}_i^{(<k)} \right\} \quad (1)$$

Type_i 字段指定三类智能体之一：(1) 编排智能体是本文提出并训练的本地部署大语言模型 (LLM)，用于以端到端方式生成多轮 YAML 交互拓扑并管理多个智能体的执行；(2) LLM 智能体是分配了角色、以提示为条件的大语言模型（开源模型或通过 API 调用）；(3) 工具智能体配备可调用的外部 API，例如检索引擎或代码执行工具。 $\text{Role}_i^{(k)}$ 是特定于该轮的角色/提示（例如 `<planner>`、`<coder>`）。 $\text{View}_i^{(k-1)}$ 是由编排智能体为该智能体整理的可见上下文，其中包含其依赖项的选定输出，也可能包含上一轮输出。最后， $\text{Mem}_i^{(<k)}$ 存储智能体 i 在第 k 轮之前的跨轮历史。

智能体通信边记号 在本框架中，边集直接由 YAML 规划中指定的 `ref` 字段构建，我们将边分为三类。第一，轮内边 $\mathcal{E}^{\text{intra}} \subseteq \mathcal{V}^{(k)} \times \mathcal{V}^{(k)}$ 根据声明的引用连接同一轮中的智能体。第二，跨轮跨智能体边 $\mathcal{E}^{\text{cross}} \subseteq \mathcal{V}^{(k-1)} \times \mathcal{V}^{(k)}$ 刻画连续两轮之间的依赖关系，即第 k 轮中的智能体显式引用第 $k-1$ 轮其他智能体的输出。第三，跨轮自边 $\mathcal{E}^{\text{self}} \subseteq \{(v_i^{(k-1)}, v_i^{(k)}) \mid v_i \in \mathcal{V}\}$ 会在同一智能体于连续两轮中被调用时自动添加，使其能够吸收并细化自身先前的输出。

编排智能体引导的多智能体交互 给定任务 x ，编排智能体输出第 k 轮的 YAML 规划。规划词元从编排策略中采样，并确定性地解码为严格分层有向无环图 $\mathcal{G}^{(k)}$ （见式 2）。节点集 $\mathcal{V}^{(k)}$ 由大语言模型智能体和工具智能体实例化；执行过程遵循 $\mathcal{G}^{(k)}$ 所隐含的步骤（层）顺序：同一步骤内的智能体并行运行，且步骤内部不存在边。我们有意排除步骤内交互，以便并行执行并降低调度复杂度。尽管全连接有向无环图具有更强的表达能力，但我们发现，强制保持步骤内部的结构稀疏性可以改善可解释性、效率和学习稳定性。对于节点 $v_i^{(k)}$ ，其第 k 轮输出生成

$$M_i^{(k)} \sim \mathcal{P}_{\theta_i}(M \mid x, \text{Role}_i^{(k)}, \text{View}_i^{(k-1)}, \text{Mem}_i^{(<k)}, \{M_j^{(k)} : (v_j^{(k)}, v_i^{(k)}) \in \mathcal{E}^{(k)}\}). \quad (14)$$

$\mathcal{E}^{(k)}$ 是从 YAML `ref` 字段解析得到的轮内依赖集（严格分层有向无环图）； $\{M_j^{(k)} : (v_j^{(k)}, v_i^{(k)}) \in \mathcal{E}^{(k)}\}$ 收集第 k 轮中 $v_i^{(k)}$ 的所有入邻居输出； $\text{Role}_i^{(k)}$ 是 v_i 特定于该轮的角色/提示； $\text{View}_i^{(k-1)}$ 是由编排智能体整理并作为只读上下文提供的上一轮摘要（拓扑/错误线索）； $\text{Mem}_i^{(<k)}$ 是第 k 轮之前该智能体局部的跨轮记忆； $\mathcal{P}_{\theta_i}$ 表示智能体的条件核（对语言智能体而言是大语言模型似然；对工具智能体而言是检索器 r 或执行器 ξ 等确定性算子）； $M_i^{(k)}$ 则是 $v_i^{(k)}$ 在第 k 轮生成的输出。

执行后，每个智能体将其输出追加到记忆中，即 $\text{Mem}_i^{(\leq k)} = \bigcup_{t=1}^k \{M_i^{(t)}\}$ 。每轮结束时，测试智能体执行候选代码并返回状态 $s^{(k)}$ ；该状态可以是 `PASSED`，或式 11 所定义集合 $\mathcal{E}_{\text{errors}}$ 中的一种错误。若 $s^{(k)} = \text{PASSED}$ ，流程停止并接受该解答。否则，编排智能体收集观测 $\mathcal{O}^{(k)} = \{\mathcal{E}_{\text{errors}}, \mathcal{L}_{\text{logs}}, \mathcal{G}^{(k)}\}$ ，其中包括错误类型 $\mathcal{E}_{\text{errors}}$ 、执行日志 $\mathcal{L}_{\text{logs}}$ 和第 k 轮拓扑轨迹 $\mathcal{G}^{(k)}$ 。编排智能体根据该观测，通过式 1 和式 2 生成下一轮交互图。在此过程中，编排智能体决定从记忆中复用哪些智能体、重新运行哪些智能体以及激活哪些智能体。编排智能体按需为每一轮持续重新生成拓扑，直至代码结果为 `PASSED`，或达到最大轮数 K 。

定义 1. 对于严格分层有向无环图 $\mathcal{G}^{(k)}$ ，节点集 $\mathcal{V}^{(k)}$ 被划分为具有明确分层结构的 b 个独立集 $\{\mathcal{V}_1^{(k)}, \dots, \mathcal{V}_b^{(k)}\}$ 。它具有以下性质：

(**顺序性**) 对任意边 (u, v) ，均有 $u \in \mathcal{V}_i^{(k)}$ 、 $v \in \mathcal{V}_j^{(k)}$ 且 $i < j$ 。

(**简洁性**) 对任意满足 $i \neq b$ 的节点 $u \in \mathcal{V}_i^{(k)}$ ，必存在边 (u, v) ，使得 $v \in \mathcal{V}_j^{(k)}$ 且 $i < j$ 。

C.1. AgentConductor 的算法流程

算法 1 汇总了 AgentConductor 的整体算法流程。

C.2. 拓扑密度的理论推导与证明

从词元成本到拓扑密度 为了实现节省成本的目标，我们依据成本效率定义拓扑密度。下面给出数学推导，说明在多智能体系统中，智能体交互的复杂度可以形式化地映射为图属性，从而量化运行成本。

首先，我们将每轮交互建模为图 $\mathcal{G}^{(k)} = (\mathcal{V}^{(k)}, \mathcal{E}^{(k)})$ ，其中顶点 $\mathcal{V}^{(k)}$ 表示智能体，边 $\mathcal{E}^{(k)}$ 刻画第 k 轮中的依赖关系。

为消除难度对拓扑规模的影响，我们考察每个智能体的平均成本。对每个智能体而言，词元成本主要由三部分构成：提示、参考信息和输出。为简化这一过程，我们作如下假设。（1）对每个智能体，提示与输出的长度相同且固定，记为 m 。（2）在第 k 轮中，必须考虑前几轮的信息。因此，假设每个智能体的输入中额外包含 $|\mathcal{V}^{(k-1)}| \times m$ 个词元。（3）在相同难度等级下，对 $\forall i, j \leq k$ ，有 $|\mathcal{V}^{(i)}| \approx |\mathcal{V}^{(j)}|$ 。

总成本可近似表示为如下形式：

$$\mathcal{C}_{\text{total}} = \sum_i^{|\mathcal{V}^{(k)}|} m + m \times |\mathcal{V}^{(k-1)}| + m \times |\text{Agent}_i[\text{ref}]| + m \times |W_{\text{ref}}(\text{Agent}_i)|, \quad (15)$$

其中， $W_{\text{ref}}(\text{Agent}_i)$ 定义为 $\{a \mid \text{Agent}_i \in a[\text{ref}]\}$ ，包含所有引用过 Agent_i 的智能体。该表达式可进一步简化为式 16：

$$\mathcal{C}_{\text{total}} = m \times (|\mathcal{V}^{(k)}| + |\mathcal{V}^{(k)}| \cdot |\mathcal{V}^{(k-1)}| + \sum_i^{|\mathcal{V}^{(k)}|} (|\text{Agent}_i[\text{ref}]| + |W_{\text{ref}}(\text{Agent}_i)|)). \quad (16)$$

注意， $\sum_i^{|\mathcal{V}^{(k)}|} |\text{Agent}_i[\text{ref}]| = \sum_i^{|\mathcal{V}^{(k)}|} |W_{\text{ref}}(\text{Agent}_i)| = |E|$ ，因此总成本由式 17 给出。

$$\mathcal{C}_{\text{total}} = m \times (|\mathcal{V}^{(k)}| + |\mathcal{V}^{(k)}| \cdot |\mathcal{V}^{(k-1)}| + 2|E|). \quad (17)$$

根据假设（3），每个智能体的平均成本由式 18 给出。

$$\bar{\mathcal{C}} = m \times (1 + |V| + 2 \frac{|E|}{|V|}). \quad (18)$$

注意，线性结构的拓扑始终具有较低的复杂度分数。然而，线性结构无法并行调用智能体，这意味着下一个智能体必须等待当前智能体完成任务，而不能同时工作。考虑到这一时间成本（也称延迟），我们将图深度 d 纳入考量。在最小化平均成本时，可以忽略常数部分和词元长度 m ，由此得到归一化前的拓扑密度表达式。

$$\mathcal{S} = |V| + 2 \frac{|E|}{|V|} + d. \quad (19)$$

由此，交互成本在分析上与以下三个拓扑特征相关：

- **智能体数量** $N = |V|$ ：智能体总数是基础计算与内存开销的主要驱动因素。每个智能体通常封装一个大语言模型（LLM）或策略网络，因此推理、状态维护和上下文管理的成本至少随 N 线性增长。这代表维护系统的固定成本。
- **边密度**：平均度 $\bar{e} = \frac{|E|}{|V|}$ 与交互开销相关。密度越高，每个节点涉及的成对交互越多，从而增加同步和消息传递成本。

- **图深度 d** : 任意两个智能体之间最长路径的节点数定义了最坏情况下的协调延迟。深度较大时需要多跳通信, 从而放大延迟和潜在的错误传播。

智能体数量和边密度可以从 YAML 字段的定义中显式导出, 但深度 d 需要额外计算。为解决这一问题, 我们提取管理智能体引导的多智能体交互所具有的性质, 并将其归纳为以下定理。

定理 1. 给定由管理智能体引导的多智能体交互所定义的有向无环图 $\mathcal{G}^{(k)}$, $\mathcal{G}^{(k)}$ 是具有 b 个分部的分部图。则有 $d^{(k)} = b$, 其中 $d^{(k)}$ 是 $\mathcal{G}^{(k)}$ 的深度。

证明. 首先证明存在一条长度为 b 的路径, 等价地, **存在一条依次访问每个分部 $V_1, V_2, \dots, V_b$ 的路径。**

根据定义, V_1 仅包含**源点** (在 $\mathcal{G}^{(k)}$ 内没有入边), 而 V_b 仅包含**汇点** (在 $\mathcal{G}^{(k)}$ 内没有出边)。任取汇点 $t \in V_b$ 。由于 $t \in V_b$ 且边从编号较小的分部指向编号较大的分部, 因此 t 必须有一个前驱 $p_{b-1} \in V_{b-1}$ (若 $b > 1$)。类似地, p_{b-1} 必须有一个前驱 $p_{b-2} \in V_{b-2}$ 。重复这一过程, 可得到一条从汇点向后追溯的路径:

$$p_1 \rightarrow p_2 \rightarrow \dots \rightarrow p_{b-1} \rightarrow t,$$

其中, 对 $i = 1, 2, \dots, b-1$, 有 $p_i \in V_i$ 。正向路径 $P = p_1 \rightarrow p_2 \rightarrow \dots \rightarrow p_{b-1} \rightarrow t$ 访问 b 个不同分部 ($V_1, V_2, \dots, V_b$), 且恰好包含 b 个顶点。

接下来证明 $d \leq b$ 。

假设存在一条包含 $m > b$ 个顶点的路径 $P = v_1 \rightarrow v_2 \rightarrow \dots \rightarrow v_m$ 。令 $v_i \in V_{a_i}$ 。由于任意边 $v_i \rightarrow v_{i+1}$ 都必须满足 $a_i < a_{i+1}$ (根据定义 1), 分部下标序列严格递增:

$$a_1 < a_2 < \dots < a_m.$$

该序列包含 m 个互不相同的整数。然而, 这些整数必须全部属于集合 $\{1, 2, \dots, b\}$, 而该集合仅包含 b 个互不相同的整数。假设 $m > b$ 意味着要在大小为 b 的集合中找到超过 b 个互不相同的整数, 这是不可能的。因此, 不存在这样的路径 P 。所以, 任意路径至多包含 b 个顶点, 深度 $d \leq b$ 。□

需要强调的是, 在大多数情况下, 智能体调用步骤满足 $s = b$, 这意味着可以直接计算 b 。然而, 在少数情况下, 两个层之间可能不发生交互, 例如 $\mathcal{V}_i^{(k)}$ 与 $\mathcal{V}_j^{(k)}$ 。此时, $\mathcal{V}_i^{(k)} \cup \mathcal{V}_j^{(k)}$ 是一个独立集, 导致 $b < s$ 并带来额外响应时间。因此, 我们使用 s 衡量图深度, 以区分拓扑相同的两个序列。

于是, 拓扑密度的基本表达式如式 20 所示:

$$\mathcal{S} = |V| + 2 \frac{|E|}{|V|} + s. \quad (20)$$

拓扑密度归一化 给定难度等级 l , 最大允许节点数为 $N_{\max}(l)$ 。为了将不同难度下的密度归一化到相同分布, 我们将该公式缩放至 $(0, 1)$ 。

首先, 有 $\frac{|V|}{N_{\max}(l)} \leq 1$ 。限制 $|V|$ 的上界后, 我们进一步约束 $\frac{|E|}{|V|}$ 。注意, 智能体通信边分为三类: 轮内边、跨轮跨智能体边和跨轮自边。其中, 根据定义 1, 轮内边满足 $|E_{\text{intra}}| \leq \frac{|V|(|V|-1)}{2}$ 。对于跨轮边, 根据假设 (3), 跨轮自边的数量可近似等于 $|V|$, 且跨轮跨智能体边满足 $|E_{\text{cross_inter}}| \leq |V|(|V|-1)$ 。因此, 对于边密度,

$$\bar{e} \leq \frac{|E_{\text{intra}}|}{|V|} + \frac{|E_{\text{self_inter}}|}{2|V|} + \frac{|E_{\text{cross_inter}}|}{2|V|}, \quad (21)$$

其简化形式为 $\bar{e} \leq |V| - 0.5$ 。因此，归一化形式为 $\frac{|E|}{|V|(|V|-0.5)}$ 。当拓扑退化为线性结构时，深度 d 等于其上界 $|V|$ ，所以有 $\frac{z}{|V|} \leq 1$ 。

复杂度越高，最终复杂度分数应当越低。因此，在式 7 中，我们使用指数函数 e^{-x} ，为复杂度分数 $S_{complexity}$ 的最终表达式实现单调递减激活函数。

C.3. 多智能体角色的详细定义

受 MapCoder 设计启发，我们的智能体池由六种不同类型的智能体组成，各自负责代码生成过程中的不同功能。每一轮代码生成中，管理智能体执行推理，并从池中选择所需智能体。图 3 中部列出了各类智能体的名称与词元表示。

C.3.1. 检索智能体

沿用 Search-R1(Jin et al., 2025)，以下检索智能体采用 E5 模型作为统一检索器。E5 充当检索骨干，在推理期间由检索智能体调用，用于识别语义相关文档。检索智能体可以将其他智能体的输入纳入参考上下文，以提高检索准确率。为了检索语义相似的代码解答，我们构建了一个离线检索智能体。沿用 VoyageAI 的做法，我们针对每个具有标准解答的基础编程问题（即 APPS、HumanEval 和 MBPP），将自然语言问题描述与相应参考实现拼接，为其创建一份文档。高级库用法。

C.3.2. 规划智能体

规划智能体以原始问题和管理智能体在上一步选定的其他智能体输出为输入，旨在生成逐步编码计划以解决原始问题。此外，规划智能体还可以根据先前的错误消息和上一轮计划迭代细化其计划，以生成更有效的求解策略。

C.3.3. 算法智能体

算法智能体以代码问题和其他智能体的输出为输入，生成针对给定问题定制算法求解步骤序列。

C.3.4. 编码智能体

编码智能体利用问题描述、规划智能体生成的逐步编码计划，以及检索智能体检索到的代码片段或教程等参考材料，生成初始代码解答。

C.3.5. 调试智能体

从第二轮开始，当初始代码生成遇到问题时，调试智能体可利用先前的错误消息与交互历史迭代修改代码。或者，它也可以根据更新后的编码计划和新检索到的参考材料重新生成代码。采用哪种具体策略由管理智能体作出的规划决策决定。

C.3.6. 测试智能体

每次迭代结束时，我们调用测试智能体评估生成代码的正确性。它返回二元通过/失败信号和分级错误诊断，这些结果既用于计算奖励函数，也作为迭代过程的终止条件。

D. 强化学习的补充定义

D.1. 强化学习中多轮轨迹与回报的定义

我们将多轮轨迹定义为：

$$\tau = \{(o_k, z_k, r_k)\}_{k=0}^{K-1}, \quad (22)$$

其中， o_k 是编码第 k 轮交互拓扑的 YAML 词元序列， z_k 表示环境产生的相应多智能体执行结果， r_k 是根据执行结果赋予的即时奖励。奖励通过函数 $r_\phi(\cdot)$ 计算，该函数评估当前交互图与代码验证结果：

$$r_k = r_\phi(\mathcal{G}^{(k)}, z_k^{\text{code}}) \quad (23)$$

其中， z_k^{code} 是在沙箱化代码验证工具中执行输入输出测试用例所得的结果。根据代码是否通过测试或遇到的具体错误类型，系统赋予不同的奖励或惩罚。此外，结构贡献根据 $\mathcal{G}^{(k)}$ 的拓扑密度是否保持在由问题难度决定的任务特定上限内计算。轨迹的整体回报定义为逐轮奖励的折扣和：

$$R(\tau) = \sum_{k=0}^{K-1} \gamma^k r_k, \quad (24)$$

其中， $\gamma \in [0, 1]$ 是折扣因子，用于调节早期奖励与后期奖励的相对重要性。该回报作为优化策略的训练信号。

D.2. 生成自适应复杂度拓扑的强化学习目标

GRPO 目标函数可形式化表示如下：

$$J_{\text{GRPO}}(\theta) = \frac{1}{G} \sum_{i=1}^G \frac{1}{L_i} \sum_{k=0}^{K_i-1} \sum_{u=1}^{|o_{i,k}|} \min \left[\frac{\pi_\theta(o_{i,k,u} \mid x, H_{i,k}, o_{i,k,<u})}{\pi_{\text{old}}(o_{i,k,u} \mid x, H_{i,k}, o_{i,k,<u})} \hat{A}_i, \right. \\ \left. \text{clip} \left(\frac{\pi_\theta(o_{i,k,u} \mid x, H_{i,k}, o_{i,k,<u})}{\pi_{\text{old}}(o_{i,k,u} \mid x, H_{i,k}, o_{i,k,<u})}, 1 - \varepsilon, 1 + \varepsilon \right) \hat{A}_i \right] - \beta \mathbb{D}_{\text{KL}}^{(\text{topo})}. \quad (25)$$

这里， $L_i = \sum_{k=0}^{K_i-1} |o_{i,k}|$ 表示轨迹 i 中拓扑词元的总数， ε 控制裁剪范围， $\mathbb{D}_{\text{KL}}^{(\text{topo})}$ 是仅在拓扑词元上计算的词元级 KL 正则项（如式 26 所示）。

基于规则的多目标奖励函数设计 奖励函数会直接影响强化学习的优化过程。本小节详细说明式 23 中即时逐轮奖励函数 $r_\phi(\cdot)$ 的定义。

一般回报 $R(\tau)$ 作为训练信号，用于优化拓扑生成策略。该策略旨在生成结构复杂度可随输入问题难度动态调整的交互图，同时最大化所生成代码通过全部测试用例的概率。我们的目标是最大化从当前策略采样所得轨迹的期望回报，同时使用词元级 Kullback–Leibler (KL) 散度相对于参考策略进行正则化。需要注意，策略 π_θ 只负责生成拓扑词元序列 o_k ；所有智能体响应与代码执行轨迹（包含在 z_k 中）均视为环境输出，不纳入 KL 正则项。

我们定义如下轨迹级优化目标：

$$\max_{\theta} \mathbb{E}_{x \sim \mathcal{D}, \{o_k\} \sim \pi_\theta} [R(\tau)] - \beta \mathbb{E}_{\{o_k\} \sim \pi_\theta} \left[\frac{1}{L(\tau)} \sum_{k=0}^{K-1} \sum_{u=1}^{|o_k|} \log \frac{\pi_\theta(o_{k,u} \mid x, H_k, o_{k,<u})}{\pi_{\text{ref}}(o_{k,u} \mid x, H_k, o_{k,<u})} \right] \quad (26)$$

其中, $\tau = \{(o_k, z_k, r_k)\}_{k=0}^{K-1}$ 是由策略 π_θ 采样所得拓扑序列 $\{o_k\}$ 诱导的轨迹, 相应交互图、智能体输出和奖励均由环境确定性地产生。 $L(\tau) = \sum_{k=0}^{K-1} |o_k|$ 表示轨迹中的拓扑词元总数, β 是平衡奖励最大化与策略偏离的权重系数。这里, x 是从数据集 $\mathcal{D}$ 抽取的问题实例, $o_{k,<u} = (o_{k,1}, \dots, o_{k,u-1})$ 表示第 k 轮中位置 u 之前生成的前缀词元序列。

D.3. 奖励设计与敏感性分析

D.3.1. 奖励设计原则

我们的奖励设计遵循三个核心目标: (1) 确保 YAML 拓扑在语法上有效; (2) 保证所生成解答的功能正确性; (3) 通过鼓励智能体拓扑具备难度感知的稀疏性来控制通信成本。这些目标通过两个分量实现: r_e 表示执行正确性 (语法和解答结果), r_g 表示拓扑密度。二者分离后, 可以分别有针对性地优化正确性与结构效率。

YAML 格式与结构有效性 无效的 YAML 结构无法支持有效的多智能体执行, 因此会获得较强的负奖励。其他 YAML 格式惩罚仅作用于拓扑结构本身, 与角色或任务无关。一旦 YAML 结构正确, 惩罚便变为零, 使 r_e 能够只关注程序执行的正确性。

难度感知密度上限 我们还针对不同难度等级的任务, 将拓扑密度上限分别设为 4、7 和 10。这些数值来自对数千个 SFT 生成样本的统计分析, 分析考察了成功求解所需拓扑密度的分布。

算法 1 AgentConductor 的在线拓扑生成流程**输入:** 输入查询 x , 策略模型 π_θ , 最大轮数 K **输出:** 最终输出 z

- 1: 初始化历史 H .
- 2: 为每个智能体 v_i 初始化局部记忆 $\{\text{Mem}_i\}$
- 3: 初始化 $z \leftarrow \emptyset$
- 4: **循环** 轮次 $k \leftarrow 1$ 到 K **执行**
- 5: $o_k = (o_{k,1}, \dots, o_{k,|o_k|}) \sim \pi_\theta(\cdot | x, H_k)$
- 6: **若** 在 o_k 中未检测到有效 YAML **则**
- 7: $y_k \leftarrow \text{YAMLCheck}(o_k)$
- 8: $H_{k+1} \leftarrow H_k.\text{append}((o_k, y_k))$
- 9: **继续**
- 10: **结束 若**
- 11: $\mathcal{G}^{(k)} = \text{DecodeTopo}(o_k)$
- 12: $z_k = (z_k^{\text{roles}}, z_k^{\text{code}}) \leftarrow \text{ExecRun}(x, \mathcal{G}^{(k)}, H_k)$
- 13: **若** z_k^{code} 中包含 PASSED **则**
- 14: 跳出 {提} 前停止
- 15: **结束 若**
- 16: $H_{k+1} \leftarrow H_k.\text{append}((\mathcal{G}^{(k)}, z_k))$
- 17: $z \leftarrow z + z_k$
- 18: **结束 循环**
- 19: **返回** 最终输出 z
- 20: **过程** $\text{ExecRun}(x, \mathcal{G}^{(k)}, H_k)$
- 21: 初始化 $z_k^{\text{roles}} \leftarrow \emptyset$
- 22: **循环** layer 属于 $\mathcal{G}^{(k)}$ **执行**
- 23: 并行运行 $\{v_i \mid v_i \in \text{layer}\}$:
- 24: $M_i^{(k)} \sim \mathcal{P}_{\theta_i}(M \mid x, \text{Role}_i^{(k)}, \text{View}_i^{(k-1)}, \text{Mem}_i^{(<k)}, \{M_j^{(k)} : (v_j^{(k)}, v_i^{(k)}) \in \mathcal{E}^{(k)}\})$
- 25: 将 $M_i^{(k)}$ 加入 Mem_i
- 26: $z_k^{\text{roles}} \leftarrow z_k^{\text{roles}} + M_i^{(k)}$
- 27: **结束 循环**
- 28: 从 z_k^{roles} 中提取代码 code_k
- 29: $z_k^{\text{code}} \leftarrow \text{tester}(\text{code}_k)$
- 30: **返回** $(z_k^{\text{roles}}, z_k^{\text{code}})$
- 31: **过程结束**